Study Notes: Days 22–28 — ASP.NET Core

These notes follow Month 1, Week 4 of the roadmap. Short ASP.NET Core examples are independent application fragments unless stated otherwise. Day 28 includes a complete local API example. The examples describe illustrative designs, not claims about Sai's actual work or FinanceHub implementation.

Day 22 — ASP.NET Core architecture

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Trace an HTTP request through an ASP.NET Core application and explain how hosting, middleware, routing, dependency injection, and application code fit together.

Hosting and request handling

ASP.NET Core is a framework for building HTTP applications. The host starts the application, manages configuration and services, and coordinates startup and shutdown. Kestrel is the usual cross-platform web server: it accepts connections and exposes requests to the ASP.NET Core pipeline. A reverse proxy or load balancer may sit in front of Kestrel to handle concerns such as TLS termination and traffic distribution.

Forwarded headers must be trusted only from configured proxies; otherwise client-supplied headers could be mistaken for the original scheme or IP address. Kestrel can also serve traffic directly in supported deployments. The host, web server, application pipeline, and deployment infrastructure are separate concepts: changing one does not automatically change all the others.

The middleware pipeline and endpoints

A request passes through an ordered pipeline of middleware. Each component may perform work before invoking the next component, after it returns, or short-circuit the pipeline. Routing selects an endpoint based on request metadata; endpoint execution invokes a minimal API handler, MVC controller action, or another endpoint type. Responses flow back outward through the middleware that called the next component.

Ordering affects correctness: authentication must establish a principal before authorization evaluates it, and exception handling must wrap the work whose exceptions it should handle. Some components need routing information before they run. Minimal hosting can insert certain middleware automatically, but explicit ordering is preferable when the order is significant. Once a response has started, an exception handler generally cannot replace its status and body as if no response had been sent.

Application layers and cross-cutting services

Endpoint code should translate HTTP input and results; application or domain code should enforce relevant business behavior; infrastructure can handle persistence and external calls. These are useful responsibility boundaries, not a requirement to create a separate project or interface for every class. A small API can stay simple while still separating validation, domain rules, and storage when that improves clarity.

Dependency injection constructs collaborators and manages registered lifetimes. Configuration supplies environment-specific settings; logging and diagnostics make operations observable. Async I/O helps a server handle more concurrent waiting requests, but does not make `DbContext` thread-safe or eliminate

downstream capacity limits. Propagate cancellation where appropriate, protect secrets, and decide explicitly how failures map to safe HTTP responses.

Code example

Create a minimal web project with `dotnet new web`, then use this as its `Program.cs`:

```
using Microsoft.AspNetCore.Http;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();

var app = builder.Build();
app.UseExceptionHandler();

app.MapGet("/health", () => Results.Ok(new { status = "healthy" }));
app.MapGet("/greet/{name}", (string name) =>
    Results.Ok(new { message = $"Hello, {name}" }));

app.Run();
```

The host builds the service container and request pipeline; routing selects a handler; the handler returns an HTTP result. This liveness-style endpoint only proves the app can answer; it does not check database or dependency readiness.

DSA problem: Maximum Subarray — Kadane's algorithm

Problem: Given a nonempty integer array, find the maximum sum of a contiguous subarray. For example, `[-2, 1, -3, 4, -1, 2, 1, -5, 4]` has maximum sum `6` from `[4, -1, 2, 1]`.

Approach: At each position, the best subarray ending there either starts at the current value or extends the best subarray ending at the previous position. Track that value and the best seen overall. Initialize from the first element so an all-negative array returns its largest element rather than an invalid empty-subarray sum of zero.

Complexity: O(n) time and O(1) additional space. The sum uses `long` to avoid `int` overflow for `int[]` inputs.

```
using System;

Console.WriteLine(MaximumSubarray(new[] { -2, 1, -3, 4, -1, 2, 1, -5, 4 })); // 6
Console.WriteLine(MaximumSubarray(new[] { -5, -2, -8 })); // -2

static long MaximumSubarray(int[] values)
{
    if (values.Length == 0)
        throw new ArgumentException("The array must not be empty.",
nameof(values));

    long endingHere = values[0];
```

```
        long best = endingHere;
        for (int i = 1; i < values.Length; i++)
        {
            endingHere = Math.Max((long)values[i], endingHere + values[i]);
            best = Math.Max(best, endingHere);
        }
        return best;
    }
```

Edge cases: A single element returns itself; all-negative arrays must not return zero unless an empty subarray is explicitly allowed; zero values and multiple equally good subarrays are valid. This version returns the sum, not the subarray indices. Empty input is explicitly rejected.

Likely interview questions

1. What are the distinct roles of the host, Kestrel, middleware, and endpoints?
2. What changes when a reverse proxy sits in front of Kestrel?
3. Why must forwarded headers be trusted only from known proxies?
4. How does a request travel into and back out of middleware?
5. Why does authentication need to run before authorization?
6. How would you separate HTTP handling, business rules, and persistence without overengineering?
7. Why is asynchronous I/O useful for server scalability?
8. Explain why Kadane's algorithm handles an all-negative array.

Common pitfalls / misconceptions

- Treating Kestrel, the host, and middleware as interchangeable terms.
- Assuming a successful liveness endpoint proves every dependency is healthy.
- Trusting forwarded headers from arbitrary clients.
- Initializing Kadane's result to zero when a nonempty subarray is required.

Cheat-sheet recap

- Host starts and manages the app; Kestrel accepts HTTP connections.
- Ordered middleware surrounds routing and endpoint execution.
- DI, configuration, logging, and cancellation support the application but do not replace sound boundaries.
- Kadane: best ending here = max(current value, previous best ending here + current value).

Day 23 — DI lifetimes: Transient, Scoped, Singleton

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Choose service lifetimes according to ownership, state, disposal, and concurrency requirements.

Transient

The container creates a transient service each time it is resolved. This is useful for lightweight, short-lived, stateless collaborators. It does not mean “one instance per method call”: if a singleton resolves and stores a transient, that particular instance can live as long as the singleton.

Disposable transient services created by the built-in container are tracked for disposal by the scope that resolved them. Resolving many such objects from the root provider can retain them until the application shuts down. Do not manually dispose a container-owned dependency; establish an appropriate scope or explicit ownership when the application must control creation and disposal.

Scoped

A scoped service is created once per DI scope and reused within that scope. ASP.NET Core normally creates a scope for each HTTP request. A scoped service is useful for request-oriented work and commonly for an EF Core `DbContext`, but request-scoped does not mean thread-safe. Concurrent operations must not use the same `DbContext` instance simultaneously.

Scopes are not necessarily HTTP requests: a background worker can explicitly create one per job or batch. Dispose that scope after the work completes. Do not let a reference escape beyond the scope or continue using a disposed dependency.

Singleton

A singleton is reused across scopes for the lifetime of the root service provider. It suits stateless, thread-safe services or shared state with appropriate synchronization. Its members can be called concurrently by multiple requests, so mutable state needs a deliberate concurrency design. Do not store request-specific state on a singleton.

A singleton must not capture a scoped service: doing so promotes request-oriented state into a longer lifetime and can cause incorrect state sharing, disposal problems, or concurrency bugs. Scope validation detects common violations when enabled, but it does not prove every dependency is safe. For background work, a singleton hosted service can inject `IServiceScopeFactory`, create a scope, resolve scoped dependencies within it, and await the work before disposing the scope.

Choosing and verifying a lifetime

Start from the dependency's actual ownership and thread-safety requirements, not a blanket rule that every service should be scoped. A transient can depend on a scoped service when resolved inside an appropriate scope. An externally created singleton instance may have different disposal ownership than an instance the container constructs. Avoid resolving arbitrary business dependencies from `IServiceProvider`; constructor injection keeps them visible.

Code example

This minimal API can be used as `Program.cs` in a `dotnet new web` project:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddTransient<TransientId>();
builder.Services.AddScoped<ScopedId>();
builder.Services.AddSingleton<SingletonId>();

var app = builder.Build();
app.MapGet("/lifetimes", (TransientId firstTransient, ScopedId firstScoped,
    SingletonId singleton, IServiceProvider services) =>
{
```

```
    var secondTransient = services.GetRequiredService<TransientId>();
    var secondScoped = services.GetRequiredService<ScopedId>();
    return Results.Ok(new
    {
        transient1 = firstTransient.Id,
        transient2 = secondTransient.Id,
        scoped1 = firstScoped.Id,
        scoped2 = secondScoped.Id,
        singleton = singleton.Id
    });
});
app.Run();

sealed class TransientId { public Guid Id { get; } = Guid.NewGuid(); }
sealed class ScopedId { public Guid Id { get; } = Guid.NewGuid(); }
sealed class SingletonId { public Guid Id { get; } = Guid.NewGuid(); }
```

Within one request, the two transient IDs differ and the scoped IDs match. The singleton ID remains the same across requests to the same running root provider. `IServiceProvider` is used here to demonstrate repeated resolution; ordinary application services should normally declare dependencies explicitly.

Likely interview questions

1. How do transient, scoped, and singleton resolutions differ?
2. Does a transient always die immediately after a method call?
3. Why is a scoped `DbContext` not automatically safe for concurrent work within a request?
4. What is a captive dependency, and why is singleton-to-scoped capture dangerous?
5. How should a background worker use a scoped service?
6. Who disposes a service created by the container, and why can root-resolved disposable transients be problematic?
7. How do you protect mutable singleton state?
8. What would you test to verify an application's registrations and lifetimes?

Common pitfalls / misconceptions

- Choosing lifetime based only on object creation cost.
- Storing request state in a singleton or capturing a scoped dependency.
- Assuming request scope serializes access to every service.
- Manually disposing a dependency owned by the container.

Cheat-sheet recap

- Transient: new per resolution; scoped: one per scope; singleton: one per root provider.
- Lifetimes govern reuse and ownership, not thread safety.
- Create explicit scopes for background units of work and dispose them after completion.

Day 24 — Middleware — request pipeline

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Predict pipeline ordering, short-circuiting, response behavior, and error handling.

How middleware composes

Middleware receives an `HttpContext` and typically a delegate representing the rest of the pipeline. It can inspect or modify the request, call the next delegate, and inspect or modify the response after the next delegate returns. Middleware is composed in registration order on the way in and unwinds in reverse order on the way out.

A component can short-circuit by returning a response without calling `next`; later middleware and endpoints then do not run. This is appropriate for cases such as a rejected request or a static file that has been served. Call `next` once in the normal path; calling it twice can execute downstream logic twice. Do not assume code after `next` runs if an exception escapes unless cleanup is in `finally`.

Ordering and production concerns

Place error handling early enough to wrap components whose exceptions it should handle. Run authentication before authorization. CORS, routing, static files, and endpoint-specific policies have ordering requirements; follow the framework's guidance for the application style and be explicit when the order matters. Static-file middleware can short-circuit, so placing it before authorization may make those files publicly accessible.

Check `Response.HasStarted` before assuming headers or status can still be changed. Do not log secrets or raw sensitive request bodies by default. Propagate cancellation using `HttpContext.RequestAborted` where downstream operations support it. Use `try/finally` for cleanup; catch exceptions only when the component can handle or translate them correctly.

Conventional middleware is constructed at startup and typically lives for the application's lifetime. A scoped dependency should be supplied to its `Invoke`/`InvokeAsync` method or used via factory-based middleware, not captured in the conventional middleware constructor. Middleware must handle concurrent requests safely.

Code example

```
using System.Diagnostics;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) =>
{
    var timer = Stopwatch.StartNew();
    try
    {
        await next(context);
    }
    finally
    {
        app.Logger.LogInformation("Request {Path} took {ElapsedMs} ms",
            context.Request.Path, timer.ElapsedMilliseconds);
    }
});
```

```
app.Use(async (context, next) =>
{
    if (context.Request.Path == "/blocked")
    {
        context.Response.StatusCode = StatusCodes.Status403Forbidden;
        await context.Response.WriteAsync("Blocked.");
        return;
    }
    await next(context);
});

app.MapGet("/hello", () => "Hello");
app.Run();
```

`/blocked` short-circuits the later pipeline, but the outer timing middleware still reaches `finally`. The example is illustrative; production logging should consider privacy, cardinality, and error reporting.

DSA problem 1: Two Sum via HashMap

Problem: Given an integer array and an integer target, return indices of two different elements that sum to the target, or `null` if no pair exists.

Approach: For each element, look for its complement among earlier elements in a dictionary, then store the current value and index. Checking first prevents using the current index twice. Use a `long` complement so subtracting extreme integers does not overflow.

Complexity: Expected O(n) time and O(n) extra space; hash-table worst cases can be worse.

```
using System;
using System.Collections.Generic;

var result = TwoSum(new[] { 3, 3 }, 6);
Console.WriteLine(result is { } pair ? $"{pair.First}, {pair.Second}" : "No
pair"); // 0, 1

static (int First, int Second)? TwoSum(int[] values, int target)
{
    var seen = new Dictionary<long, int>();
    for (int i = 0; i < values.Length; i++)
    {
        long complement = (long)target - values[i];
        if (seen.TryGetValue(complement, out int earlier))
            return (earlier, i);
        seen[values[i]] = i;
    }
    return null;
}
```

Edge cases: Empty or one-element arrays have no pair; duplicates can form a valid pair; negative values and zero work. Any valid pair is returned, not necessarily every pair.

DSA problem 2: Frequency counting

Problem: Count the number of occurrences of each integer in an input array.

Approach: Use a dictionary from value to count; increment the existing count or initialize it to one.

Complexity: Expected O(n) time and O(k) extra space for `k` distinct values.

```
using System;
using System.Collections.Generic;

var counts = CountFrequencies(new[] { 4, 2, 4, 4, 2 });
Console.WriteLine(counts[4]); // 3
Console.WriteLine(counts[2]); // 2

static Dictionary<int, int> CountFrequencies(int[] values)
{
    var counts = new Dictionary<int, int>();
    foreach (int value in values)
    {
        counts.TryGetValue(value, out int count);
        counts[value] = count + 1;
    }
    return counts;
}
```

Edge cases: Empty input returns an empty dictionary; repeated, negative, and zero values work. Do not rely on dictionary iteration order when an ordered output is required.

Likely interview questions

1. What happens before and after `next` in a middleware pipeline?
2. What does short-circuiting mean, and what downstream work is skipped?
3. Why must exception handling wrap the components whose failures it handles?
4. Why can a conventional middleware constructor not safely capture a scoped service?
5. What can and cannot be changed after a response starts?
6. How can middleware ordering accidentally expose static files?
7. Why does the Two Sum algorithm check before inserting?
8. What are the time and space tradeoffs of frequency counting with a dictionary?

Common pitfalls / misconceptions

- Calling the rest of the pipeline more than once.
- Putting critical middleware after a component that can short-circuit.
- Logging sensitive data indiscriminately or trying to rewrite an already-started response.
- Reusing the same index in Two Sum or depending on dictionary enumeration order.

Cheat-sheet recap

- Middleware runs in registration order inward and reverse order outward.
- Short-circuiting skips downstream components.
- Order exception handling, routing, authentication, and authorization deliberately.
- Two Sum uses complement lookup; frequency counting maps each key to its count.

Day 25 — Routing + Controllers

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Explain how a request selects an endpoint and how controllers translate HTTP requests into application operations.

Routing

Endpoint routing matches request properties such as path and HTTP method to a registered endpoint. Route templates can include literals, parameters, optional segments, and constraints. For example, `{id:guid}` constrains a segment to GUID-shaped input. Constraints help disambiguate routes; they are not a replacement for business validation. A request that does not match the constraint can produce a 404 rather than a model-validation 400.

Route design should be predictable and use resource-oriented paths where appropriate. Avoid ambiguous templates and conflicting HTTP methods. Link generation and named routes can reduce hard-coded URLs, but generated links must respect the application's proxy and host configuration.

Controllers and actions

Controllers group related HTTP actions and work well with MVC's filters, model binding, validation, and response conventions. Minimal APIs are a lighter alternative for focused endpoints; neither style automatically produces a better architecture. Keep controllers thin: receive and validate HTTP input, call application behavior, and map results to appropriate responses. Business rules and database queries need not be embedded in the action.

`[ApiController]` enables useful API-specific behavior, including automatic 400 responses for model-state validation failures and binding-source inference in many cases. It requires attribute routing for actions. Route attributes such as `[HttpGet(\"{id:guid}\")]` specify the HTTP method and template. Use asynchronous actions for asynchronous I/O and pass cancellation through when supported.

Responses and correctness

A `GET` of a missing resource commonly returns 404; a successful creation commonly returns 201 with a `Location` pointing to the created resource; a successful deletion can return 204. A route constraint only checks route shape, not whether the resource exists or the caller may access it. Authentication and authorization remain separate checks; avoid exposing information about resources across tenants.

`ActionResult<T>` allows a typed successful result alongside other HTTP responses. The exact status and response body should be part of the API contract, not inferred from a controller method name. Unhandled failures should be logged and converted to safe error responses at the appropriate boundary.

Code example

For a controller-based web application, register and map controllers:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
var app = builder.Build();
app.MapControllers();
app.Run();
```

Add this controller in the same project:

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/greetings")]
public sealed class GreetingsController : ControllerBase
{
    [HttpGet("{id:guid}")]
    public ActionResult<GreetingResponse> Get(Guid id)
    {
        if (id == Guid.Empty) return BadRequest("Id must not be empty.");
        return Ok(new GreetingResponse(id, "Hello"));
    }
}

public sealed record GreetingResponse(Guid Id, string Message);
```

This example intentionally generates a response rather than fetching a real resource. A production action would call an appropriate application service and handle absence and authorization.

Likely interview questions

1. How does endpoint routing select an action?
2. What is the difference between a route constraint and business validation?
3. What behavior does `[ApiController]` add?
4. When might controllers be preferable to minimal APIs, or vice versa?
5. Why should a controller generally avoid owning all domain and persistence logic?
6. What do `ActionResult<T>` and `CreatedAtAction` help express?
7. What status should be returned for malformed input versus a missing resource?
8. How do route matching and authorization differ?

Common pitfalls / misconceptions

- Treating a route constraint as complete validation.
- Putting resource authorization or business rules solely in route templates.
- Returning success for missing data or exposing unhandled internal errors.

- Assuming minimal APIs and controllers require fundamentally different hosting pipelines.

Cheat-sheet recap

- Routing selects an endpoint from path, method, and related metadata.
- Controllers adapt HTTP to application operations; `[ApiController]` adds API conventions.
- Constraints disambiguate routes; validation and authorization enforce other rules.

Day 26 — Model binding + validation

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Explain how incoming HTTP data becomes typed input, distinguish malformed input from invalid business state, and return useful validation errors.

Model binding and input formatting

Model binding obtains values from sources such as route parameters, query strings, form fields, and request bodies and converts them into action parameters or models. In controller APIs, input formatters deserialize JSON request bodies. Attributes such as `[FromRoute]`, `[FromQuery]`, and `[FromBody]` make the source explicit when useful; `[ApiController]` can infer many sources. Binding rules depend on the application style and parameter type, so verify ambiguous cases rather than assuming every parameter comes from the same place.

Conversion can fail before business logic runs—for example, a nonnumeric value for an integer or malformed JSON. In controllers, errors are recorded in model state; `[ApiController]` normally turns invalid model state into an automatic 400 response. Minimal APIs have their own binding behavior and do not automatically apply all MVC data-annotation validation in every framework version; check the target version and configure validation explicitly.

Validation

Validation checks whether a successfully bound input satisfies rules. Data annotations such as `[Required]`, `[StringLength]`, and `[Range]` can express simple field constraints. Cross-field rules such as a maturity date not preceding an opening date need an appropriate validator or explicit domain check. Database constraints should protect invariants that must hold across concurrent requests; an application-level check alone cannot prevent races.

Differentiate transport validation from domain and authorization rules. A syntactically valid request can still violate a business invariant or be forbidden for the current user. Avoid binding persistence entities directly from untrusted input: dedicated request DTOs reduce accidental overposting of fields such as owner ID or administrative status. Define consistent, safe error responses without leaking database or internal exception details.

Code example

In a controller application with `AddControllers()` and `MapControllers()`:

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/deposits")]
public sealed class DepositsController : ControllerBase
{
    [HttpPost]
    public IActionResult Create([FromBody] CreateDepositRequest request)
    {
        if (request.MaturesOn < request.OpenedOn)
        {
            ModelState.AddModelError(nameof(request.MaturesOn),
                "Maturity date must not precede the opening date.");
            return ValidationProblem(ModelState);
        }

        // Illustrative response only; no deposit is persisted here.
        return Ok(new { request.Institution, request.Principal });
    }
}

public sealed class CreateDepositRequest
{
    [Required, StringLength(100)]
    public string Institution { get; init; } = "";

    [Range(typeof(decimal), "0.01", "1000000000000")]
    public decimal Principal { get; init; }

    [Required]
    public DateOnly? OpenedOn { get; init; }

    [Required]
    public DateOnly? MaturesOn { get; init; }
}
```

With `[ApiController]`, missing required dates and invalid annotated fields produce a 400 before the action runs. The cross-field check runs in the action. A real endpoint must persist the result, decide currency and rounding rules, and authorize the caller.

DSA problem: First non-repeating character

Problem: Given a string, return the index of its first character that occurs exactly once, or `-1` if none exists. Define “character” here as a C# `char` (a UTF-16 code unit), with case-sensitive comparison.

Approach: Count each code unit in a dictionary, then scan the string from the beginning and return the first index whose count is one. Two passes preserve original order without depending on dictionary iteration order.

Complexity: Expected O(n) time and O(k) extra space for `k` distinct code units.

```
using System;
using System.Collections.Generic;

Console.WriteLine(FirstUniqueIndex("swiss")); // 1: 'w'
Console.WriteLine(FirstUniqueIndex("aabb"));  // -1

static int FirstUniqueIndex(string text)
{
    var counts = new Dictionary<char, int>();
    foreach (char character in text)
    {
        counts.TryGetValue(character, out int count);
        counts[character] = count + 1;
    }
    for (int i = 0; i < text.Length; i++)
        if (counts[text[i]] == 1) return i;
    return -1;
}
```

Edge cases: Empty input returns `-1`; comparison is case-sensitive; whitespace is counted like any other code unit. `char` is not always a complete Unicode character: supplementary characters use surrogate pairs, and user-perceived characters may contain multiple code points. For international text, agree whether indices and uniqueness refer to UTF-16 units, Unicode scalar values (`Rune`), or grapheme clusters, and handle normalization deliberately.

Likely interview questions

1. What are common binding sources for an HTTP request?
2. How do model binding failures differ from domain validation failures?
3. What does `[ApiController]` do with invalid model state?
4. Why might a minimal API need explicit validation?
5. How would you validate a rule involving two dates?
6. How do DTOs reduce overposting risks?
7. Why can an application-level uniqueness check still race?
8. What does “character” mean in .NET when solving first-non-repeating-character problems?

Common pitfalls / misconceptions

- Assuming route, query, and body values all use the same binding rules.
- Trusting annotations alone for cross-request or database invariants.
- Binding untrusted input directly into privileged persistence fields.
- Treating UTF-16 `char` values as universally equivalent to user-perceived characters.

Cheat-sheet recap

- Bind and convert input, validate it, authorize the operation, then enforce domain and persistence invariants.
- `[ApiController]` commonly returns 400 automatically for invalid model state.

- First unique code unit: count, then scan in input order; define Unicode semantics.

Day 27 — Configuration + Options pattern + logging

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Supply environment-specific configuration safely, consume related settings through typed options, and produce useful structured logs.

Configuration

The default host combines configuration providers such as JSON files, environment variables, and command-line arguments. Later providers generally override earlier ones for the same key. Environment variables commonly use `__` to represent nested keys, such as `Pricing__MaxAmount`. Inspect the actual provider ordering when customizing the host.

Separate environment-specific settings from application code. Do not commit credentials or secrets to source control; use development secret storage and a suitable production secret manager or platform mechanism. Configuration is input: validate required values and ranges at startup where possible. Changing a configuration source does not mean every service automatically observes a new value.

Options pattern

The options pattern binds related settings to a typed object. `IOptions<T>` provides a value for the application lifetime; `IOptionsSnapshot<T>` is scoped and computes values per scope, useful when supported configuration changes should appear on later requests; `IOptionsMonitor<T>` is singleton-compatible and can expose updated values and change notifications when the underlying provider supports reload.

Neither snapshot nor monitor guarantees a provider will reload, and changes can occur while work is in progress. For a multi-step operation, capture a consistent options value at its start if that is the intended behavior. Validate options, including relationships between fields, before serving traffic. Do not inject a scoped snapshot into a singleton.

Logging

`ILogger<T>` records events through configured providers. Structured templates such as `\"Processed order {OrderId}\"` preserve named properties for search and aggregation; string interpolation generally loses that structure. Log at a level appropriate to the event, add an exception object when recording a failure, and avoid logging the same exception repeatedly at every layer.

Logs are not a substitute for metrics or traces. Include correlation or trace context where supported, but avoid high-cardinality or sensitive data such as credentials, account numbers, and raw request bodies unless a justified and controlled policy permits it. Logging can have cost and retention implications; never assume a log write proves a business transaction committed.

Code example

This uses built-in options binding in a minimal web application. Set `Limits__MaxItems=20` as an environment variable or configure `Limits:MaxItems` in a JSON provider.

```
using Microsoft.Extensions.Options;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddOptions<LimitOptions>()
    .Bind(builder.Configuration.GetSection("Limits"))
    .Validate(o => o.MaxItems > 0, "Limits:MaxItems must be positive.")
    .ValidateOnStart();

var app = builder.Build();
app.MapGet("/limit", (IOptions<LimitOptions> options, ILogger<Program> logger) =>
{
    int maxItems = options.Value.MaxItems;
    logger.LogInformation("Configured maximum items: {MaxItems}", maxItems);
    return Results.Ok(new { maxItems });
});
app.Run();

sealed class LimitOptions
{
    public int MaxItems { get; set; } = 10;
}
```

`10` is an explicit example default. The validation rejects nonpositive configured values at startup. For settings that must always be supplied, validate their presence instead of providing a default.

Likely interview questions

1. How are conflicting configuration values resolved across providers?
2. How do nested configuration keys map to environment variables?
3. Where should secrets live in development and production?
4. Compare `IOptions<T>`, `IOptionsSnapshot<T>`, and `IOptionsMonitor<T>`.
5. Does `IOptionsMonitor<T>` guarantee that every provider reloads?
6. How would you validate required configuration before serving requests?
7. Why prefer structured logging templates over interpolated strings?
8. How do you avoid leaking sensitive data while preserving useful diagnostics?

Common pitfalls / misconceptions

- Committing secrets or assuming configuration values are always trustworthy.
- Capturing a scoped options snapshot in a singleton.
- Assuming every setting reloads automatically or that a reload is atomic across an entire operation.
- Logging sensitive request data or treating logs as proof of committed business state.

Cheat-sheet recap

- Configuration combines providers; later providers generally override earlier ones.
- Use typed options with validation; choose lifetime and reload behavior deliberately.
- Use structured logs, appropriate levels, correlation, and privacy controls.

Day 28 — Build a small Finance API from scratch: GET/POST/PUT/DELETE /api/fds — not tutorial-following

Month 1 — Foundations, Week 4 — ASP.NET Core.

Goal: Design and build a small persistent Finance API independently, then identify the extra safeguards required for actual FinanceHub use.

Build exercise and contract

Before looking at the worked example, design the API from memory: define the resource and request DTO, validation rules, storage boundary, DI registration, routes, status codes, and error handling. Then build and exercise all endpoints. Here **FD** is an explicit example assumption meaning a fixed deposit; adjust the fields and rules to match the actual FinanceHub idea rather than assuming this example describes it.

The local example supports:

Method	Route	Behavior
GET	`/api/fds`	Return the stored deposits.
GET	`/api/fds/{id}`	Return one deposit, or 404.
POST	`/api/fds`	Validate and create; return 201 and a `Location` header.
PUT	`/api/fds/{id}`	Validate and replace the editable fields; return 204 or 404.
DELETE	`/api/fds/{id}`	Delete; return 204 or 404.

This is a persistent **local development baseline**, not a production finance service. It uses SQLite for durability across restarts and parameterized SQL for values, but has no authentication, authorization, tenant ownership, encryption, audit trail, pagination, optimistic concurrency, backups, or deployment hardening. In particular, concurrent updates are last-write-wins. Do not deploy it for real financial data without designing and implementing those controls.

Worked example: persistent local API

With the .NET 10 SDK installed, create the application and add the SQLite provider:

```
dotnet new web -n FinanceApi
cd FinanceApi
dotnet add package Microsoft.Data.Sqlite --version 10.0.0
```

Replace the generated `Program.cs` with the following. Run `dotnet run`; the listening address is printed in the console. The local database file is `finance.db` in the process working directory unless a `ConnectionStrings:Finance` configuration value is supplied. Configuration and the database file are local to the application, not the browser.

```
using System.Globalization;
using Microsoft.Data.Sqlite;
```

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
string connectionString = builder.Configuration.GetConnectionString("Finance")
    ?? "Data Source=finance.db";
builder.Services.AddScoped(_ => new FdStore(connectionString));

var app = builder.Build();
app.UseExceptionHandler();
FdStore.Initialize(connectionString);

var routes = app.MapGroup("/api/fds");
routes.MapGet("/", (FdStore store) => Results.Ok(store.List()));
routes.MapGet("/{id:guid}", (Guid id, FdStore store) =>
{
    var fd = store.Find(id);
    return fd is null ? Results.NotFound() : Results.Ok(fd);
});
routes.MapPost("/", (FdInput input, FdStore store) =>
{
    var errors = Validate(input);
    if (errors.Count != 0) return Results.ValidationProblem(errors);
    var fd = ToFd(Guid.NewGuid(), input);
    store.Add(fd);
    return Results.Created($"/api/fds/{fd.Id}", fd);
});
routes.MapPut("/{id:guid}", (Guid id, FdInput input, FdStore store) =>
{
    var errors = Validate(input);
    if (errors.Count != 0) return Results.ValidationProblem(errors);
    return store.Replace(ToFd(id, input)) ? Results.NoContent() :
Results.NotFound();
});
routes.MapDelete("/{id:guid}", (Guid id, FdStore store) =>
    store.Delete(id) ? Results.NoContent() : Results.NotFound());

app.Run();

static Dictionary<string, string[]> Validate(FdInput input)
{
    var errors = new Dictionary<string, string[]>();
    if (string.IsNullOrWhiteSpace(input.Institution) ||
input.Institution.Trim().Length > 100)
        errors["institution"] = new[] { "Institution is required and must be at
most 100 characters." };
    if (input.Currency is null || input.Currency.Length != 3 ||
        !input.Currency.All(c => c >= 'A' && c <= 'Z'))
        errors["currency"] = new[] { "Currency must be three uppercase ASCII
letters." };
    if (input.Principal is not decimal principal || principal <= 0 ||
        principal > 1_000_000_000_000m || decimal.Round(principal, 2) !=
principal)
        errors["principal"] = new[] { "Principal must be positive, at most
1000000000000, and have at most two decimal places." };
```

```
    if (input.AnnualRatePercent is not decimal rate || rate < 0 || rate > 100 ||
        decimal.Round(rate, 4) != rate)
        errors["annualRatePercent"] = new[] { "Rate must be between 0 and 100 with
at most four decimal places." };
    if (input.OpenedOn is null) errors["openedOn"] = new[] { "Opening date is
required." };
    if (input.MaturesOn is null) errors["maturesOn"] = new[] { "Maturity date is
required." };
    if (input.OpenedOn is DateOnly opened && input.MaturesOn is DateOnly matures
&& matures < opened)
        errors["maturesOn"] = new[] { "Maturity date must not precede opening
date." };
    return errors;
}

static Fd ToFd(Guid id, FdInput input) => new(
    id, input.Institution!.Trim(), input.Currency!, input.Principal!.Value,
    input.AnnualRatePercent!.Value, input.OpenedOn!.Value,
input.MaturesOn!.Value);

public sealed record FdInput(
    string? Institution,
    string? Currency,
    decimal? Principal,
    decimal? AnnualRatePercent,
    DateOnly? OpenedOn,
    DateOnly? MaturesOn);

public sealed record Fd(
    Guid Id,
    string Institution,
    string Currency,
    decimal Principal,
    decimal AnnualRatePercent,
    DateOnly OpenedOn,
    DateOnly MaturesOn);

public sealed class FdStore
{
    private readonly string connectionString;
    public FdStore(string connectionString) => this.connectionString =
connectionString;

    public static void Initialize(string connectionString)
    {
        using var connection = new SqliteConnection(connectionString);
        connection.Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            CREATE TABLE IF NOT EXISTS fds (
                id TEXT PRIMARY KEY,
                institution TEXT NOT NULL,
                currency TEXT NOT NULL,
                principal TEXT NOT NULL,
```

```
                annual_rate_percent TEXT NOT NULL,
                opened_on TEXT NOT NULL,
                matures_on TEXT NOT NULL
            );
            """;
        command.ExecuteNonQuery();
    }

    public List<Fd> List()
    {
        using var connection = Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            SELECT id, institution, currency, principal, annual_rate_percent,
opened_on, matures_on
            FROM fds ORDER BY id;
            """;
        using var reader = command.ExecuteReader();
        var result = new List<Fd>();
        while (reader.Read()) result.Add(Read(reader));
        return result;
    }

    public Fd? Find(Guid id)
    {
        using var connection = Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            SELECT id, institution, currency, principal, annual_rate_percent,
opened_on, matures_on
            FROM fds WHERE id = $id;
            """;
        command.Parameters.AddWithValue("$id", id.ToString());
        using var reader = command.ExecuteReader();
        return reader.Read() ? Read(reader) : null;
    }

    public void Add(Fd fd)
    {
        using var connection = Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            INSERT INTO fds (id, institution, currency, principal,
annual_rate_percent, opened_on, matures_on)
            VALUES ($id, $institution, $currency, $principal, $rate, $opened,
$matures);
            """;
        AddParameters(command, fd);
        command.ExecuteNonQuery();
    }

    public bool Replace(Fd fd)
    {
        using var connection = Open();
```

```
        using var command = connection.CreateCommand();
        command.CommandText = """
            UPDATE fds SET institution = $institution, currency = $currency,
                principal = $principal, annual_rate_percent = $rate,
                opened_on = $opened, matures_on = $matures
            WHERE id = $id;
            """;
        AddParameters(command, fd);
        return command.ExecuteNonQuery() == 1;
    }

    public bool Delete(Guid id)
    {
        using var connection = Open();
        using var command = connection.CreateCommand();
        command.CommandText = "DELETE FROM fds WHERE id = $id;";
        command.Parameters.AddWithValue("$id", id.ToString());
        return command.ExecuteNonQuery() == 1;
    }

    private SqliteConnection Open()
    {
        var connection = new SqliteConnection(connectionString);
        connection.Open();
        return connection;
    }

    private static void AddParameters(SqliteCommand command, Fd fd)
    {
        command.Parameters.AddWithValue("$id", fd.Id.ToString());
        command.Parameters.AddWithValue("$institution", fd.Institution);
        command.Parameters.AddWithValue("$currency", fd.Currency);
        command.Parameters.AddWithValue("$principal",
fd.Principal.ToString(CultureInfo.InvariantCulture));
        command.Parameters.AddWithValue("$rate",
fd.AnnualRatePercent.ToString(CultureInfo.InvariantCulture));
        command.Parameters.AddWithValue("$opened", fd.OpenedOn.ToString("yyyy-MM-
dd", CultureInfo.InvariantCulture));
        command.Parameters.AddWithValue("$matures", fd.MaturesOn.ToString("yyyy-
MM-dd", CultureInfo.InvariantCulture));
    }

    private static Fd Read(SqliteDataReader reader) => new(
        Guid.Parse(reader.GetString(0)),
        reader.GetString(1),
        reader.GetString(2),
        decimal.Parse(reader.GetString(3), CultureInfo.InvariantCulture),
        decimal.Parse(reader.GetString(4), CultureInfo.InvariantCulture),
        DateOnly.ParseExact(reader.GetString(5), "yyyy-MM-dd",
CultureInfo.InvariantCulture),
        DateOnly.ParseExact(reader.GetString(6), "yyyy-MM-dd",
CultureInfo.InvariantCulture));
}
```

`decimal` values are stored as invariant-culture text to avoid binary floating-point representation in this simple schema. Text storage has different sorting and aggregation semantics from numeric storage; do not perform numerical SQL comparisons or sums on these text columns without an appropriate schema/design. The three-letter currency check validates shape, not membership in an authoritative currency list. The code uses synchronous SQLite operations for clarity; measure and choose the appropriate persistence technology and access model for the workload.

Exercise the API

Replace `BASE` with the URL printed by `dotnet run`, for example `http://localhost:5000`. Create a record:

```
curl -i -X POST "$BASE/api/fds" \
  -H 'Content-Type: application/json' \
  -d '{"institution":"Example
Bank","currency":"INR","principal":10000.00,"annualRatePercent":7.25,"openedOn":"2
026-01-01","maturesOn":"2027-01-01"}'
```

Set `ID` to the `id` from the response, then exercise the other methods:

```
curl -i "$BASE/api/fds"
curl -i "$BASE/api/fds/$ID"
curl -i -X PUT "$BASE/api/fds/$ID" \
  -H 'Content-Type: application/json' \
  -d '{"institution":"Example
Bank","currency":"INR","principal":10000.00,"annualRatePercent":7.5,"openedOn":"20
26-01-01","maturesOn":"2027-01-01"}'
curl -i -X DELETE "$BASE/api/fds/$ID"
curl -i "$BASE/api/fds/$ID"
```

Expected statuses: create 201, list and existing get 200, successful update and delete 204, then get 404. Test a negative principal and reversed dates for 400 validation responses, an invalid GUID route for 404, malformed JSON for 400, and update/delete of a missing GUID for 404. Create a record, restart the app, and verify it remains present before deleting it. A local `BASE` assignment is required before these commands, for example `BASE=http://localhost:5000`; use the port actually printed by your app.

Real-world tie-in: FinanceHub

The roadmap connects this exercise to the FinanceHub idea: build the real thing rather than treating CRUD as an isolated toy. Reflect on the fields and workflows **you actually intend**: who owns each deposit, what currency and maturity rules apply, how updates are authorized, and what must be auditable. Separate implemented features from planned ones.

Model explanation: “FinanceHub needs [actual user and workflow]. This API currently supports [verified endpoints and storage]. To make it suitable for that workflow, I would add [specific missing control] because [risk], and verify it with [test]. The current example does not yet provide [honest limitation].” Adapt this to your real idea; do not claim production use or measured outcomes that have not occurred.

Likely interview questions

1. How would you design the request and response contracts for the deposit endpoints?
2. Why should input DTOs be separate from persisted or privileged fields?
3. Which status codes should create, update, delete, missing-resource, and validation paths return?
4. How would you prevent one user from reading or changing another user's deposit?
5. What happens when two clients update the same record, and how would optimistic concurrency address it?
6. Why are decimal precision, currency, and rounding rules important for finance data?
7. How would you test persistence, validation, error handling, and restart behavior?
8. What additional reliability and security work is required before this example handles real financial data?

Common pitfalls / misconceptions

- Treating working CRUD endpoints as production-ready financial software.
- Ignoring ownership checks, concurrency conflicts, auditability, backups, and secrets.
- Using binary floating point without explicit financial precision and rounding decisions.
- Reporting the sample's assumptions as actual FinanceHub features or results.

Cheat-sheet recap

- Design the contract and business rules first; implement and exercise every endpoint.
- Validate inputs, use parameterized persistence, and return explicit HTTP outcomes.
- Persistence alone is not enough: add authorization, concurrency control, observability, and operational safeguards.
- State clearly what is implemented, what is planned, and what remains unverified.